

Dokumentacja techniczna - Koder-Dekoder Steganograficzny z Modułem Szyfrowania

Projekt zespołowy PZ-2026 Temat: *Implementacja koder-dekoder steanograficznego z modułem wielometodowego szyfrowania danych.*

Niniejszy dokument opisuje architekturę, sposób uruchomienia, API backendu, format nagłówka metadanych oraz strukturę frontendu aplikacji, wraz z opisem wymagań funkcjonalnych i podziału zadań w zespole.

Spis treści

- Przegląd
- Architektura
- Wymagania i uruchomienie
- Przepływ danych (kodowanie i dekodowanie)
- API backendu
- Format nagłówka metadanych
- Frontend
- Struktura katalogów
- Testy

1. Przegląd

Głównym zadaniem aplikacji jest **steganografia** - ukrywanie wiadomości tekstowych w nośnikach multimedialnych. Szyfrowanie nie jest osobnym modułem aplikacji, lecz **krokiem przygotowawczym**, koniecznym przed ukryciem wiadomości: tekst najpierw szyfrowany jest jednym z algorytmów, a dopiero potem ukrywany w nośniku. Zapewnia to dwustopniowe zabezpieczenie danych:

- Szyfrowanie (krok przygotowawczy)** - tekst jest szyfrowany jednym z 7 zaimplementowanych algorytmów.
- Steganografia (główne zadanie)** - zaszyfrowany tekst jest ukrywany w najmniej znaczących bitach (LSB) nośnika multimedialnego: pliku **BMP** (obraz) lub **WAV** (dźwięk).

Parametry kodowania (użyty szyfr, liczba bitów na kanał/próbkę, tryb rozmieszczenia, długość danych) zapisywane są w dedykowanym **nagłówku metadanych** wstrzykiwanym do pliku nośnika. Dzięki temu dekodér potrafi **automatycznie rozpoznać** sposób zakodowania pliku i odtworzyć pierwotny tekst.

Obsługiwane algorytmy szyfrowania

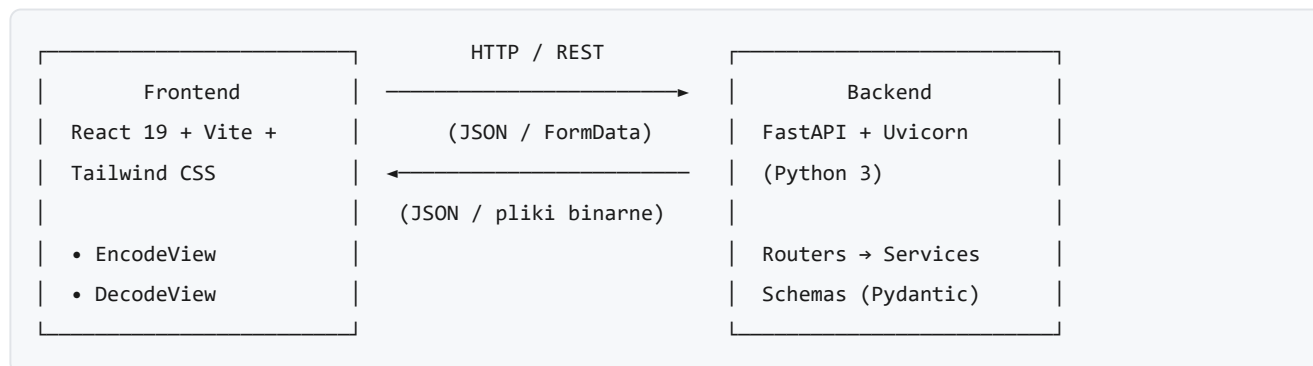
Szyfr	Wymaga klucza	Typ klucza
Szyfr Cezara	tak	liczba (przesunięcie)
Szyfr Vigenère'a	tak	tekst (słowo-klucz)
Szyfr XOR	tak	tekst
Szyfr Atbash	nie	-
ROT13	nie	-
Szyfr płotkowy (Rail Fence)	tak	liczba (liczba szyn)
Szyfr kolumnowy	tak	tekst (słowo-klucz)

Tryby rozmieszczenia danych

- **Ciągły (0)** - bity wiadomości zapisywane od początku obszaru danych nośnika.
- **Równomierny (1)** - bity rozłożone równomiernie w całej objętości pliku (dynamiczny krok).

2. Architektura

Projekt stosuje separację logiki biznesowej (Core) od interfejsu użytkownika (GUI):



Backend (`backend/`) zorganizowany jest warstwowo:

- **routers/** - definicje endpointów HTTP (FastAPI `APIRouter`), walidacja żądań, obsługa błędów.
- **services/** - logika biznesowa: algorytmy szyfrów, operacje LSB na BMP/WAV, kodowanie/odczyt nagłówka.
- **schemas/** - modele danych (Pydantic) opisujące żądania i odpowiedzi.

Frontend (`frontend/`) to aplikacja SPA z dwiema głównymi zakładkami (Koder / Dekoder), komunikująca się z backendem przez warstwę `src/api/steganographyApi.js` .

Backend domyślnie nasłuchuje na `http://127.0.0.1:3000` ; CORS jest otwarty na wszystkie źródła (`allow_origins=["*"]`).

2.1 Podział na moduły i mapowanie na kod

Prace zespołu podzielone są na **6 modułów funkcjonalnych**. Poniższa tabela mapuje każdy moduł na odpowiadające mu pliki w repozytorium oraz osoby odpowiedzialne (zgodnie z arkuszem *Podział na moduły.xlsx*):

Moduł	Zakres	Główne pliki	Zespół
Moduł 1 - Kryptografia	7 algorytmów szyfrowania/deszyfrowania	backend/app/routers/*Cipher_router.py , rot13_router.py , backend/app/services/*Cipher_service.py , rot13_service.py	cały zespół projektowy (patrz tabela poniżej)
Moduł 2 - Silnik steganograficzny	Zapis/odczyt bitów LSB w BMP i WAV	backend/app/services/steganography_service.py , backend/app/routers/steganography_router.py	Michał Krok, Oktawian Majerczak
Moduł 3 - Algorytmy rozmieszczenia	Tryb ciągły i równomierny (dynamiczny krok)	logika uniform / continuous w steganography_service.py	Adam Zadrozny, Rafał Leszczyk
Moduł 4 - Metadane i nagłówki	Kodowanie/odczyt nagłówka + hash kontrolny	backend/app/services/headerBMP_service.py , headerWAV_service.py , backend/app/routers/header_router.py , backend/app/schemas/header_schema.py	Krzysztof Guzik, Michał Bej
Moduł 5 - Dekoder	Analiza pliku, identyfikacja parametrów, deszyfrowanie	backend/app/services/decoder_service.py , backend/app/routers/decoder_router.py	Kamil Urbański, Bogumił Wójcik
Moduł 6 - GUI	Interfejs użytkownika, podgląd i wizualizacja nośnika	cały frontend/	Kacper Mielańczyk, Michał Wojtala, Włodzimierz Palazanov, Karolina Oleśniewicz

Moduł 1 (Kryptografia) realizowany był przez **cały zespół projektowy** - poszczególne szyfry przydzielono dwuosobowym grupom. Szczegółowy podział, który szyfr realizowała która para, przedstawia poniższa tabela:

Nr	Metoda szyfrowania	Opis zadania	Osoba 1	Osoba 2
1	Szyfr Cezara	Przesunięcie znaków o stałą wartość klucza.	Bogumił Wójcik	Kamil Urbański
2	Szyfr Vigenère'a	Szyfr polialfabetyczny wykorzystujący słowo-klucz.	Rafał Leszczyk	-
3	Szyfr XOR	Operacja logiczna bitowego XOR na znakach tekstu z wykorzystaniem klucza.	Michał Bej	Krzysztof Guzik
4	Szyfr Atbash	Prosty szyfr podstawieniowy polegający na odwróceniu alfabetu (A↔Z, B↔Y).	Karolina Oleśniewicz	Włodzimierz Palazanov
5	ROT13	Specyficzny wariant szyfru Cezara z przesunięciem o 13 pozycji.	Michał Wojtala	Kacper Mielańczyk
6	Szyfr płotkowy (Rail Fence)	Szyfr transpozycyjny polegający na zygzakowatym zapisie tekstu.	Adam Zadrozny	-
7	Szyfr kolumnowy	Transpozycja tekstu na podstawie klucza ustalającego kolejność kolumn w macierzy.	Oktawian Majerczak	Michał Krok

Moduły 2–6 odpowiadają kolejnym warstwom przepływu kodowania/dekodowania opisanego w sekcji 4.

3. Wymagania i uruchomienie

Wymagania

- **Python 3** (backend)
- **Node.js + Yarn** (frontend)

Backend

```
cd backend
python3 -m pip install -r requirements.txt
uvicorn main:app --port 3000 --reload
```

Zależności (backend/requirements.txt): fastapi , pydantic (v2), uvicorn , python-multipart , pytest , httpx .

Po uruchomieniu dostępne są:

- Bazowy URL: `http://127.0.0.1:3000`
- Interaktywna dokumentacja Swagger UI: `http://127.0.0.1:3000/docs`
- ReDoc: `http://127.0.0.1:3000/redoc`

Plik `main.py` zawiera również blok `if __name__ == "__main__":` uruchamiający serwer przez `python main.py` (host `127.0.0.1`, port `3000`).

Frontend

```
cd frontend
yarn install
yarn dev
```

Adres backendu można nadpisać zmienną środowiskową `VITE_API_BASE_URL` (domyślnie `http://127.0.0.1:3000`).

Skrypty (`frontend/package.json`): `yarn dev` , `yarn build` , `yarn preview` , `yarn lint` .

Pliki testowe

Repozytorium zawiera przykładowe nośniki: `test/image/*.bmp` , `test/audio/*.wav` oraz pliki pomocnicze w katalogu głównym (`test.bmp` - do zakodowania, `stego.bmp` / `final.bmp` - zakodowane).

4. Przepływ danych

Kodowanie (Koder)

1. Użytkownik wpisuje tekst, wybiera szyfr i podaje klucz (wymagany przez większość szyfrów; pomijany jedynie dla szyfrów bezkluczowych Atbash i ROT13).
2. Frontend wywołuje endpoint szyfrowania → otrzymuje zaszyfrowany tekst.
3. Frontend wstrzykuje **nagłówek metadanych** do pliku nośnika (`/header/inject-bmp/` lub `/header/inject-wav/`).
4. Frontend ukrywa zaszyfrowany tekst w pliku metodą LSB (`/steganography/hide`).
5. Użytkownik pobiera gotowy plik z ukrytą wiadomością.

Dekodowanie (Dekoder)

Dekodowanie jest **deterministyczne** - opiera się na odczytanym nagłówku. Dostępne są dwie ścieżki:

A. Ścieżka backendowa (`/decoder/process`) - pojedynczy endpoint realizujący cały proces po stronie serwera (`decoder_service.process_file`): 1. Odczyt nagłówka z pliku (szyfr, suwaki, liczba bitów, tryb). 2. Ekstrakcja zaszyfrowanego ciągu bitów (LSB). 3. Deszyfrowanie odpowiednim algorytmem z użyciem klucza podanego przez użytkownika.

B. Ścieżka frontendowa (`decodeEncodedFile`) - frontend orkiestruje kolejne wywołania: odczyt nagłówka → przywrócenie nośnika → ekstrakcja LSB → deszyfrowanie. Zwraca też przywrócony, „czysty” plik nośnika.

Uwaga: deszyfrowanie wymaga podania **tego samego klucza**, którym zaszyfrowano wiadomość (poza szyframi bezkluczowymi Atbash i ROT13). `decoder_service` zawiera dodatkowo pomocniczą funkcję `_derive_key_from_sliders` , wyprowadzającą klucz z wartości suwaków, gdy klucz nie jest dostarczony jawnie.

5. API backendu

Wszystkie endpointy są pogrupowane tagami w Swagger UI: **Cipher, Steganography, Header, Decoder Module**.

5.1 Szyfry (tag: Cipher)

Endpointy szyfrów przyjmują JSON i zwracają JSON. Pole wyniku to zwykle `output`.

Szyfr	Szyfrowanie	Deszyfrowanie	Parametry żądania
Cezar	POST /caesar/encrypt	POST /caesar/decrypt	text, shift
Vigenère	POST /vinegre/encrypt	POST /vinegre/decrypt	text, key
XOR	POST /encrypt/xor	POST /decrypt/xor	text, key
Atbash	POST /atbash/encrypt	POST /atbash/decrypt	text
ROT13	POST /rot13/process	POST /rot13/process (ten sam)	text
Rail Fence	POST /railfence/encrypt	POST /railfence/decrypt	text, rails
Kolumnowy	POST /columnar/encrypt	POST /columnar/decrypt	text, key

Przykład (XOR):

```
POST /encrypt/xor
Content-Type: application/json

{ "text": "Tajna wiadomość", "key": "SECRET" }
```

```
{ "output": "..." }
```

5.2 Steganografia (tag: Steganography)

Wszystkie endpointy `hide / extract` przyjmują `multipart/form-data`.

GET /steganography/info

Zwraca opis modułu.

POST /steganography/hide

Ukrywa zaszyfrowaną wiadomość w pliku (zwraca plik binarny BMP/WAV).

Pole	Typ	Opis
file	plik	Nośnik BMP lub WAV
encrypted_message	str	Zaszyfrowana wiadomość do ukrycia
media_type	str	"bmp" lub "wav"
deployment_mode	int	0 - ciągły, 1 - równomierny
sliderR, sliderG, sliderB	int (0–8)	Liczba bitów na kanał (BMP)
slider	int (0–8)	Liczba bitów na próbkę (WAV)

POST /steganography/extract

Ekstrahuje (wciąż zaszyfrowaną) wiadomość. Zwraca JSON { "message": "... " }. Pola jak wyżej, dodatkowo total_bits (wymagane w trybie równomiernym - całkowita liczba ukrytych bitów).

5.3 Nagłówek (tag: Header)

Endpoint	Opis
POST /header/inject-bmp/	Wstrzykuje 8-bajtowy nagłówek przed danymi pikseli BMP; zwraca zmodyfikowany plik.
POST /header/extract-bmp/	Usuwa nagłówek i przywraca oryginalny BMP.
POST /header/extract-bmp-header/	Odczytuje parametry z nagłówka BMP, zwraca JSON (cipher , sliders , bits , deployment_mode).
POST /header/inject-wav/	Analogicznie dla WAV.
POST /header/extract-wav/	Przywraca oryginalny WAV.
POST /header/extract-wav-header/	Odczytuje parametry z nagłówka WAV.

Parametry wstrzyknięcia (multipart/form-data): cipher (nazwa szyfru), bits (długość danych w bitach), deployment_mode , oraz suwaki - sliderR/G/B (BMP) lub slider (WAV).

5.4 Dekoder (tag: Decoder Module)

POST /decoder/process

Kompletny proces dekodowania po stronie serwera. multipart/form-data :

Pole	Typ	Opis
file	plik	Zakodowany nośnik
media_type	str	"bmp" lub "wav"
key	str	Klucz deszyfrowania (jeśli szyfr go wymaga)

Odpowiedź (DecoderResponse):

```
{
  "status": "success",
  "message_detected": true,
  "cipher_used": "Szyfr XOR",
  "deployment_mode": "Ciągłe",
  "bits_extracted": 1234,
  "decrypted_text": "...
}
```

Gdy plik nie zawiera ukrytej wiadomości (lub nagłówek/hash jest nieprawidłowy), zwracany jest `HTTP 400`.

6. Format nagłówka metadanych

Nagłówek opisuje, **jak** plik został zakodowany, i jest wstrzykiwany do struktury pliku nośnika (dla BMP - tuż przed danymi pikseli, z odpowiednią korektą pól rozmiaru i offsetu w nagłówku BMP). Implementacja:

`headerBMP_service.py`, `headerWAV_service.py`.

Układ bitowy (BMP)

Nagłówek BMP zajmuje **8 bajtów (64 bity)**, spakowane w kolejności little-endian:

Bity	Pole	Rozmiar	Znaczenie
0–2	<code>cipher</code>	3 bity	Indeks szyfru (enum <code>Cipher</code> , 7 wartości)
3–6	<code>slider[0]</code> (R)	4 bity	Liczba bitów kanału R (0–8)
7–10	<code>slider[1]</code> (G)	4 bity	Liczba bitów kanału G (0–8)
11–14	<code>slider[2]</code> (B)	4 bity	Liczba bitów kanału B (0–8)
15–49	<code>bits</code>	35 bitów	Długość danych w bitach (maks. 34 359 738 367)
50	<code>deployment_mode</code>	1 bit	0 ciągły / 1 równomierny
51–63	<code>hash</code>	13 bitów	Suma kontrolna (skrótowy MD5)

Pierwsze 51 bitów (7 bajtów) niesie właściwe metadane; pozostałe 13 bitów to **hash kontrolny** (pierwsze 13 bitów MD5 z 7 bajtów metadanych). Przy odczycie hash jest weryfikowany - jego niezgodność oznacza, że plik **nie zawiera** prawidłowo zakodowanej wiadomości (`extract_bmp_header_from_file` zgłasza wtedy `ValueError`). Dzięki temu dekodery potrafi automatycznie wykryć „czyste” pliki.

Zakres pola `bits` (35 bitów) wynika z tego, że długość danych obrazu w nagłówku BMP zapisana jest na 4 bajtach; długość wyrażona w bitach to maksymalnie $0xFFFFFFFF * 8$.

Nagłówek WAV

Analogiczny, lecz z jednym suwakiem (`slider`) zamiast trójki RGB - patrz `WAVHeader` w `header_schema.py`.

7. Frontend

Aplikacja React (Vite, React 19, Tailwind CSS 4). Punkt wejścia: `src/App.jsx` - dwie zakładki przełączane stanem `activeTab`:

- **Koder** - `views/EncodeView.jsx`
- **Dekoder** - `views/DecodeView.jsx`

Warstwy

Katalog	Rola
<code>src/api/</code>	Komunikacja z backendem (<code>steganographyApi.js</code>) - szyfrowanie, hide/extract, nagłówki, dekodowanie, normalizacja parametrów.
<code>src/config/</code>	Konfiguracja: <code>appConfig.js</code> (URL API, typy mediów, tryby), <code>ciphers.js</code> (definicje szyfrów, wymagane klucze).
<code>src/hooks/</code>	Logika wielokrotnego użytku: <code>useEncoder</code> , <code>useDecoder</code> , <code>useMediaFile</code> , <code>useWaveform</code> .
<code>src/components/</code>	Komponenty UI, w tym <code>common/</code> (Button, Alert, FileDropzone, SliderControl, MediaPreview, WaveformCanvas/Modal) i podgląd nośnika.
<code>src/layout/</code>	Szkielet strony: <code>AppLayout</code> , <code>Header</code> , <code>ContentWrapper</code> .
<code>src/utils/</code>	Pomocnicze: <code>detectMediaType</code> , <code>formatFileSize</code> , <code>validateCipherKey</code> .
<code>src/i18n.js</code>	Teksty interfejsu (UI_TEXT).

Kluczowa logika API (frontend)

`steganographyApi.js` mapuje nazwy szyfrów na endpointy backendu (`buildEncryptionPayload` / `buildDecryptionPayload`), normalizuje parametry (typ mediów, tryb rozmieszczenia, wartości suwaków 0–8) i udostępnia m.in.:

- `encryptText` / `decryptText`
- `hideSteganography` / `extractSteganography`
- `injectHeader` / `extractHeader` / `restoreCarrierFile`
- `decodeEncodedFile` (alias `decodeFile`) - pełny przepływ dekodowania po stronie klienta.

Wizualizacja nośnika „przed i po” realizowana jest m.in. przez `MediaPreview` oraz `WaveformCanvas` / `WaveformComparison` (oscylogram WAV).

8. Struktura katalogów

```
PZ-2026/
├─ backend/
│   ├── main.py                # aplikacja FastAPI, montaż routerów, CORS
│   ├── requirements.txt
│   └─ app/
│       ├── routers/          # endpointy: szyfry, steganografia, header, decoder
│       ├── services/         # logika: algorytmy, LSB, nagłówki
│       └─ schemas/           # modele Pydantic (żądania/odpowiedzi)
│   └─ tests/                 # testy pytest
├─ frontend/
│   ├── package.json
│   ├── vite.config.js
│   └─ src/
│       ├── api/ config/ hooks/ components/ layout/ utils/ views/
│       ├── App.jsx main.jsx i18n.js
│       └─ ...
├─ test/                     # przykładowe pliki BMP/WAV
├─ README.md                 # opis projektu, wymagania, podział zadań
└─ DOCUMENTATION.md          # ten dokument
```

9. Testy

Testy backendu (pytest + httpx `TestClient`) znajdują się w `backend/tests/` :

Plik	Zakres
<code>test_decoder_router.py</code> , <code>test_decoder_service.py</code>	Moduł dekodera (router i logika).
<code>test_header_router.py</code> , <code>test_headerBMP_service.py</code> , <code>test_headerWAV_service.py</code>	Wstrzykiwanie/odczyt nagłówka BMP i WAV.
<code>test_steganography_router.py</code> , <code>test_steganography_service.py</code>	Ukrywanie i ekstrakcja LSB.
<code>test_xorCipher_router.py</code> , <code>test_xorCipher_service.py</code>	Szyfr XOR.

Uruchomienie:

```
cd backend
python3 -m pytest
```

Pokrycie testami jest selektywne - kompletne testy istnieją dla modułów dekodera, nagłówka, steganografii oraz szyfru XOR.